BCSE204L Design and Analysis of Algorithms

Module-1

Topic-2: Divide and Conquer Strategy

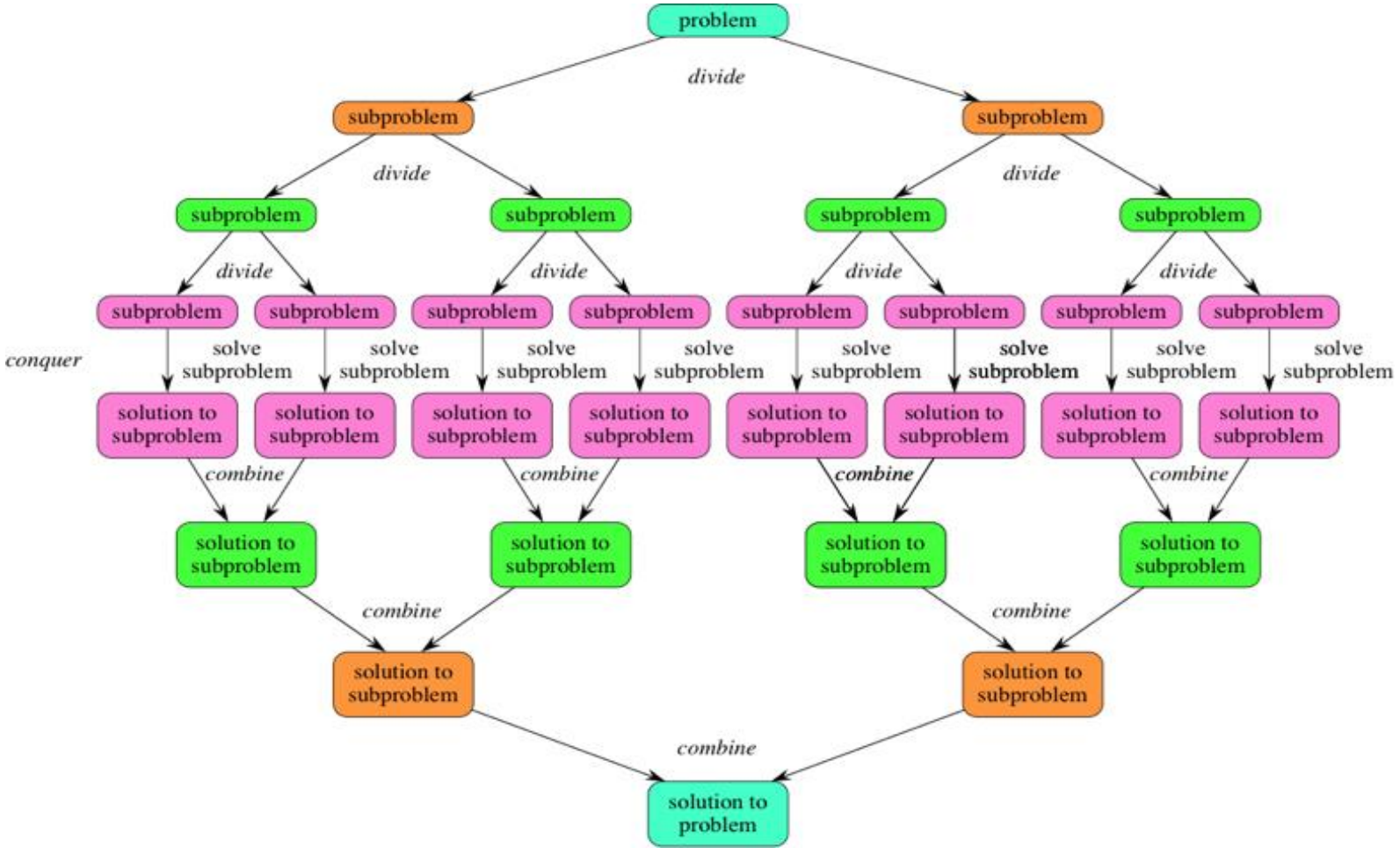
Dr Athira K
SCOPE

D&C Strategy

- Partitions the problem into disjoint subproblems, solve the subproblems recursively, and then combine their solutions to solve the original problem.
- Given a function to compute on n inputs the divide-and-conquer strategy suggests splitting the inputs into k distinct subsets,$1 < k < n$, yielding k subproblems
- Solve the k subproblems
- Find a method to combine the sub-solutions to a solution of the whole problem
- If the subproblems are still relatively large, then the divide-and-conquer strategy can possibly be reapplied (recursive)

D&C Features

- Sub-problem must be same as the problem
- Recursive in nature



D&C Approach

```
1   Algorithm DAndC(P)
2   {
3       if Small(P) then return S(P);
4       else
5       {
6           divide P into smaller instances P_1, P_2, ..., P_k, k ≥ 1;
7           Apply DAndC to each of these subproblems;
8           return Combine(DAndC(P_1),DAndC(P_2),...,DAndC(P_k));
9       }
10  }
```

D&C Approach [cont..]

$$T(n) = \begin{cases} g(n) & n \text{ small} \\ T(n_1) + T(n_2) + \cdots + T(n_k) \quad + \quad f(n) & \text{otherwise} \end{cases}$$

- $g(n)$ is the time to compute the answer directly for small inputs.
- $f\ (n)$ is the time for dividing P and combining the solutions of the subproblem

D&C Analysis

- The recurrence of many divide and conquer algorithms is given by:

$$T(n) = \begin{cases} T(1) & n = 1 \\ aT(n/b) + f(n) & n > 1 \end{cases}$$

- Where a and b are known constants. We assume that $T(1)$ is known and n is a power of b (i.e. $n = b^k$)
- Can be solved using substitution method, recurrence tree method or master method

D&C Examples

- Merge sort
- Quick sort
- Binary search

Maximum Subarray Problem

- Given an integer array, find the maximum sum among all subarrays possible
- Subarrays are required to occupy consecutive positions within the original array (Don’t confuse with subsequences)
- All positive → Entire array
- All negative → The maximum value (Size=1)

Maximum subarray problem

- Eg:- [2, -4, 1, 9, -6, 7, -3], sum = 11
- A naive solution is to consider every possible subarray, find the sum, and take the maximum.
- Worst-case time complexity is $O(n^2)$, where n is the size of the input

Better Method: D&C

- Divide the array into two equal subarrays.
- Recursively calculate the maximum subarray sum for the left subarray,
- Recursively calculate the maximum subarray sum for the right subarray,
- Find the maximum subarray sum that crosses the middle element.
- Return the maximum of the above three sums

Demo

- On board 3 -1 -1 10 -3 -2 4

Pseudocode

```
MSS(A,low,high)
if high==low// base case
   return A[low]
else
   mid=(low + high)/2
   leftsum=MSS(A,low,mid)
   rigthsum=MSS(A,mid+1,high)
   midcrosssum=CSS(A,low,mid,high)
return maximum(leftsum,rightsum,midcrosssum)
End function
```

Pseudocode [cont..]

```
CSS(A,low,mid,high)
sum=0
leftsum=-INFINITY;
for i=mid downto low
    sum=sum + A[i]
    if sum> leftsum
        leftsum=sum
sum=0
rightsum=-INFINITY;
for i=mid+1 to high
    sum= sum + A[i]
    if sum > rightsum
        rightsum= sum
return leftsum + rightsum
End function
```

Exercise

- On Moodle

Thank you..!!